Ferramenta Baseada em Linguagem de Domínio Específico para o Apoio ao Ensino de Autômatos Finitos com Simulação Visual Interativa

João Thomaz Vieira Klaus Siegfried Beckmann

Bacharelado em Ciência da Computação
Universidade Tuiuti do Paraná
Orientador: Prof. Diógenes Cogo Furlan

Curitiba, 2026

Roteiro

1. Introdução
2. Autômatos Finitos e Linguagens Formais
3. Compiladores e Linguagens de Domínio Específico
4. Rust como Linguagem de Implementação
5. Trabalhos Relacionados
6. Projeto da DSL
7. Implementação
8. Resultados
9. Conclusão

Roteiro

1. Introdução
2. Autômatos Finitos e Linguagens Formais
3. Compiladores e Linguagens de Domínio Específico
4. Rust como Linguagem de Implementação
5. Trabalhos Relacionados
6. Projeto da DSL
7. Implementação
8. Resultados
9. Conclusão

Contexto e motivação

- O estudo de **autômatos finitos e linguagens formais** é um dos pilares da formação em Ciência da Computação (máquinas abstratas, limites da computação, compiladores).
- Apesar da importância, é uma disciplina **reconhecidamente desafiadora**.
- Segundo Morazán e Antunez (2014), a dificuldade vem de uma causa precisa:

  > *o estudante projeta autômatos e prova sua correção **sem poder experimentar** nem obter **retroalimentação imediata**, ao contrário do que ocorre na programação.*

- Há um distanciamento entre o **formalismo simbólico** e a **prática computacional**.

O problema: as ferramentas existentes

Ferramentas de apoio (como o **JFLAP**) adotam uma abordagem **exclusivamente visual** (arrastar e soltar). Isso traz limitações relevantes:

- A descrição do autômato **não é textual** $\rightarrow$ dificulta versionamento (Git) e compartilhamento.
- **Não há validação formal** da estrutura com mensagens de erro claras e localizadas.
- O estudante que erra na construção **não recebe um diagnóstico preciso** do problema.

Lacuna

Falta uma ferramenta que una o **rigor da descrição textual** ao **diagnóstico de um compilador** e à **interatividade da simulação visual**.

A proposta: Autosim

Uma ferramenta educacional que acopla diretamente:

DSL textual

Linguagem de domínio específico, declarativa e em português, para descrever AFDs e AFNs.

Interface gráfica interativa

Renderiza o diagrama a partir do texto e **anima a simulação passo a passo**.

Compilador pedagógico

Análises léxica, sintática e semântica, com **diagnósticos localizados** (linha e coluna).

Contribuição central

Acoplamento direto entre a descrição textual e a geração automática de uma interface gráfica sincronizada.

Objetivos

Objetivo geral: desenvolver uma ferramenta educacional que integra a descrição textual de autômatos, sua compilação pedagógica e sua simulação visual em um único ambiente.

Objetivos específicos

1. Projetar uma **DSL** textual para descrever AFDs e AFNs (gramática em EBNF);
2. Implementar um **compilador pedagógico** (léxico, sintático e semântico) com diagnósticos localizados;
3. Desenvolver um **simulador** de cadeias sobre os autômatos descritos;
4. Construir uma **interface gráfica** que renderize o diagrama e anime a simulação;
5. **Validar** a ferramenta com autômatos clássicos e cenários de erro induzido.

Roteiro

1. Introdução
2. Autômatos Finitos e Linguagens Formais
3. Compiladores e Linguagens de Domínio Específico
4. Rust como Linguagem de Implementação
5. Trabalhos Relacionados
6. Projeto da DSL
7. Implementação
8. Resultados
9. Conclusão

Autômatos finitos

Um **AFD** é uma quíntupla $M = (Q, \Sigma, \delta, q_0, F)$:

- Q — conjunto finito de estados;
- Σ — alfabeto de entrada;
- $\delta : Q \times \Sigma \to Q$ — função de transição;
- $q_0 \in Q$ — estado inicial;
- $F \subseteq Q$ — estados de aceitação.

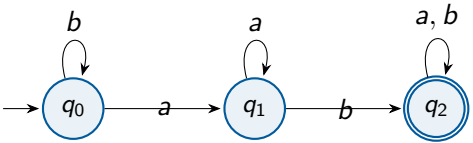
No **AFN**, a transição leva a um *conjunto* de estados ($\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to \mathcal{P}(Q)$), admitindo transições vazias ε.

Linguagem reconhecida

$$L(M) = \{\, w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F \,\}$$

AFD e AFN são **equivalentes** em poder de reconhecimento (construção de subconjuntos) e definem exatamente as **linguagens regulares**.

Autômatos finitos — um exemplo prático



Cada elemento de $M = (Q, \Sigma, \delta, q_0, F)$ no grafo:

- $Q = \{q_0, q_1, q_2\}$ — os círculos;
- $\Sigma = \{a, b\}$ — os rótulos das arestas;
- q_0 — estado inicial (seta de entrada);
- $F = \{q_2\}$ — estados finais (círculo duplo);
- δ — as próprias arestas do grafo:

δ	a	b
q_0	q_1	q_0
q_1	q_1	q_2
q_2	q_2	q_2

Reconhece as cadeias sobre Σ que **contêm "*ab*" como subcadeia**. **Aceita:** `ab`, `aab`, `bab`. **Rejeita:** ε, `ba`, `bbb`.
Reaparece adiante — na DSL e na simulação.

Gramáticas e a Hierarquia de Chomsky

Restrições sobre a forma das regras definem classes de linguagens:

Tipo	Classe	Forma das regras	Modelo
0	Recursiv. enumeráveis	$\alpha \to \beta$	Máquina de Turing
1	Sensíveis ao contexto	$\alpha A \beta \to \alpha \gamma \beta$	Autômato lin. limitado
2	Livres de contexto	$A \to \gamma$	Autômato com pilha
3	**Regulares**	$A \to aB \mid A \to a$	**Autômato finito**

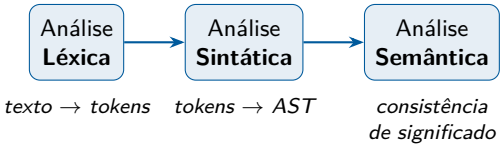
As **gramáticas regulares (Tipo 3)** geram exatamente a classe reconhecida pelos **autômatos finitos** — o formalismo central deste trabalho.

Roteiro

1. Introdução
2. Autômatos Finitos e Linguagens Formais
3. Compiladores e Linguagens de Domínio Específico
4. Rust como Linguagem de Implementação
5. Trabalhos Relacionados
6. Projeto da DSL
7. Implementação
8. Resultados
9. Conclusão

Compiladores: as fases de análise

Um compilador traduz um programa-fonte verificando sua consistência em fases sucessivas:



- **Léxica:** agrupa caracteres em unidades léxicas (*tokens*) — via expressões regulares.
- **Sintática:** organiza os *tokens* em uma **Árvore Sintática Abstrata (AST)**.
- **Semântica:** verifica regras que a gramática não captura (referências, consistência).

Linguagens de Domínio Específico (DSL)

Definição (Fowler, 2010)

Uma DSL é "uma linguagem de programação de **expressividade limitada**, voltada para um **domínio específico**".

- Ganha **concisão e legibilidade** no seu domínio, ao custo da generalidade.
- **DSL interna:** embutida em uma linguagem hospedeira.
- **DSL externa:** sintaxe própria, exige um **compilador dedicado**.

*A Autosim é uma **DSL externa*** — com sintaxe própria e um compilador implementado em Rust.

Roteiro

1. Introdução
2. Autômatos Finitos e Linguagens Formais
3. Compiladores e Linguagens de Domínio Específico
4. Rust como Linguagem de Implementação
5. Trabalhos Relacionados
6. Projeto da DSL
7. Implementação
8. Resultados
9. Conclusão

Por que Rust?

- Linguagem de sistemas com **desempenho de C/C++** e **segurança de memória** verificada em tempo de compilação (*ownership* e *borrowing*, sem coletor de lixo).
- **Segurança de tipos** facilita representar *tokens*, nós de AST e estados de autômatos sem inconsistências em tempo de execução.
- Estilo **funcional** (*traits*, iteradores) favorece compor as fases de análise de forma clara e modular.
- Ecossistema maduro (**Cargo**) com bibliotecas para *lexing*, *parsing* e GUI.

Roteiro

1. Introdução
2. Autômatos Finitos e Linguagens Formais
3. Compiladores e Linguagens de Domínio Específico
4. Rust como Linguagem de Implementação
5. Trabalhos Relacionados
6. Projeto da DSL
7. Implementação
8. Resultados
9. Conclusão

Panorama das ferramentas existentes

- **JFLAP** — referência mundial; gráfica (arrastar e soltar); cobre toda a hierarquia de Chomsky. *Sem* representação textual, versionamento ou diagnóstico.
- **GAM** (UFLA) — autômatos finitos e expressões regulares; *desktop* (C++/GTK); limitado a 10 estados e 3 símbolos.
- **Simulador UFSC** — aplicação *web* (AFD/AFN, pilha e Turing); elimina barreiras de instalação, mas mantém a construção puramente gráfica.
- **AutomatoGraph** — foco em AFDs; **gera a gramática regular** correspondente; simulação apenas parcialmente visual.
- **Análise Léxica em Rust** (Bruzzone, 2023) — rigor de engenharia (lexer via autômatos), mas *sem* suporte educacional / interface para iniciantes.

Análise comparativa

Trabalho	DSL textual	Versiona-mento	Compilador/ diagnóstico	Simulação visual	Voltada ao ensino
Análise Léxica em Rust	Não	Sim	Não	Não	Não
JFLAP	Não	Não	Não	Sim	Sim
GAM	Não	Não	Não	Sim	Sim
Simulador UFSC	Não	Não	Não	Sim	Sim
AutomatoGraph	Não	Não	Não	Parcial	Sim
Autosim (este trabalho)	**Sim**	**Sim**	**Sim**	**Sim**	**Sim**

Nenhuma ferramenta une **descrição textual + compilador com diagnóstico + simulação visual**.

É essa lacuna que a Autosim ocupa.

Roteiro

1. Introdução
2. Autômatos Finitos e Linguagens Formais
3. Compiladores e Linguagens de Domínio Específico
4. Rust como Linguagem de Implementação
5. Trabalhos Relacionados
6. Projeto da DSL
7. Implementação
8. Resultados
9. Conclusão

A linguagem: um exemplo

DSL externa, sintaxe em **português**; cada arquivo `.asl` tem um alfabeto comum e um ou mais autômatos, seguidos de comandos de simulação.

```
alfabeto { 'a', 'b' }

automato AFD exemplo_afd {
    estados { q0, q1, q2 }
    inicial q0
    finais { q2 }
    transicoes {
        q0 -> q1 com 'a'      q1 -> q2 com 'b'
        q0 -> q0 com 'b'      q1 -> q1 com 'a'
        q2 -> q2 com 'a'      q2 -> q2 com 'b'
    }
}

simular exemplo_afd com "ab"      // aceita
simular exemplo_afd com "bbb"     // rejeita
```

Gramática formal (EBNF)

Linguagem **livre de contexto**, analisável por *parser* descendente preditivo com um símbolo de *lookahead*:

```
programa            ::= declaracao_alfabeto item*
declaracao_alfabeto ::= "alfabeto" "{" [ char_literal ("," char_literal)* [","] ] "}"
item                ::= declaracao_automato | comando_simulacao
declaracao_automato ::= "automato" tipo_automato ident "{"
                            "estados" "{" [ ident ("," ident)* [","] ] "}"
                            "inicial" ident
                            "finais"  "{" [ ident ("," ident)* [","] ] "}"
                            "transicoes" "{" transicao* "}" "}"
tipo_automato       ::= "AFD" | "AFN"
transicao           ::= ident "->" ident "com" (char_literal | "epsilon" | "eps")
comando_simulacao   ::= "simular" ident "com" cadeia_literal
```

11 palavras reservadas, *case-insensitive*.

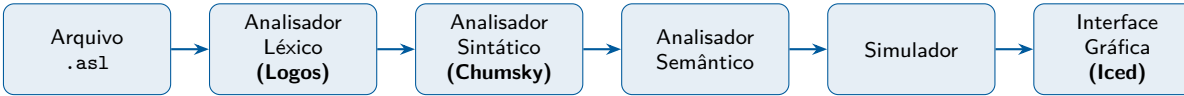
Decisões de projeto

- **Declaração única de alfabeto** — reforça o conceito teórico e simplifica a verificação semântica (um único conjunto de referência).
- **Classificação explícita AFD/AFN** — permite verificações específicas por tipo (ex.: determinismo só para AFD).
- **Sintaxe em português** — reduz a barreira linguística e mantém o foco nos conceitos de autômatos.
- **Case-insensitive** — `AFD`, `afd`, `Afd` são equivalentes; reduz a carga cognitiva do estudante.
- **Suporte a transições** ε — palavras-chave `epsilon` / `eps` para AFNs com movimentos vazios.

Roteiro

1. Introdução
2. Autômatos Finitos e Linguagens Formais
3. Compiladores e Linguagens de Domínio Específico
4. Rust como Linguagem de Implementação
5. Trabalhos Relacionados
6. Projeto da DSL
7. Implementação
8. Resultados
9. Conclusão

Pipeline de compilação



- Fases executadas **sequencialmente**; em caso de erro, os diagnósticos são exibidos (via **Ariadne**) e a execução é interrompida antes da fase seguinte.
- Implementação em **Rust**, reaproveitando bibliotecas maduras do ecossistema.

Análise léxica e sintática

Léxica — Logos

- *Lexer* gerado a partir de anotações no tipo `Token`.
- Gera um AFD para reconhecimento eficiente dos padrões.
- Palavras reservadas *case-insensitive*; espaços e comentários (`//`) descartados.
- Erro léxico $\rightarrow$ posição (linha/coluna) do caractere inválido.

Sintática — Chumsky

- *Parser combinators*: a gramática é codificada compondo funções.
- Produz a **AST**, eliminando ruídos sintáticos (delimitadores).
- *Spans* referenciam índices de *token* para localizar erros.

Análise semântica — 3 fases estáticas

1. **Tabela de símbolos:** detecta símbolo/estado/autômato **duplicado**.
2. **Verificação de referências:** estado inicial/final/origem/destino não declarado; símbolo fora do alfabeto; simulação de autômato inexistente.
3. **Verificação de determinismo** (só para AFD):
   - ausência de transições ε;
   - unicidade de transição por par (estado, símbolo).

Os erros são **acumulados** e apresentados em conjunto — o estudante vê todos os problemas de uma só vez.

Simulador

- Executa a função de transição sobre a cadeia, registrando **todas as configurações intermediárias**.
- Exibe um **caminho único** do autômato, e não a nuvem de estados simultâneos:
  - cadeia aceita $\rightarrow$ o **caminho aceitador**;
  - cadeia rejeitada $\rightarrow$ o caminho que **mais se aproxima** da aceitação.
- Escolha por **busca em largura** sobre pares (estado, símbolos consumidos): maximiza os símbolos lidos e, no empate, minimiza a distância a um estado final.
- Guarda o **histórico completo** $\Rightarrow$ navegação **bidirecional** (avançar / retroceder) durante a execução.

Interface gráfica (Iced)

- **Renderização automática** do diagrama de estados a partir do texto-fonte.
- **Simulação passo a passo**, com reprodução automática (800 ms/passo): destaca o estado corrente, a transição percorrida e a cadeia consumida.
- **Navegação** do diagrama: *pan* (arrastar) e *zoom* (rolar), com botão **Centralizar**.
- Painel muda de cor conforme o **veredito**: **verde (aceita)** / **vermelho (rejeita)**.

Diagnóstico de erros (Ariadne)

- Toda mensagem inclui **linha e coluna**, um **trecho do código com realce colorido** e uma **seta** apontando para o ponto do erro.
- Descrição do problema **em português**.

Exemplos

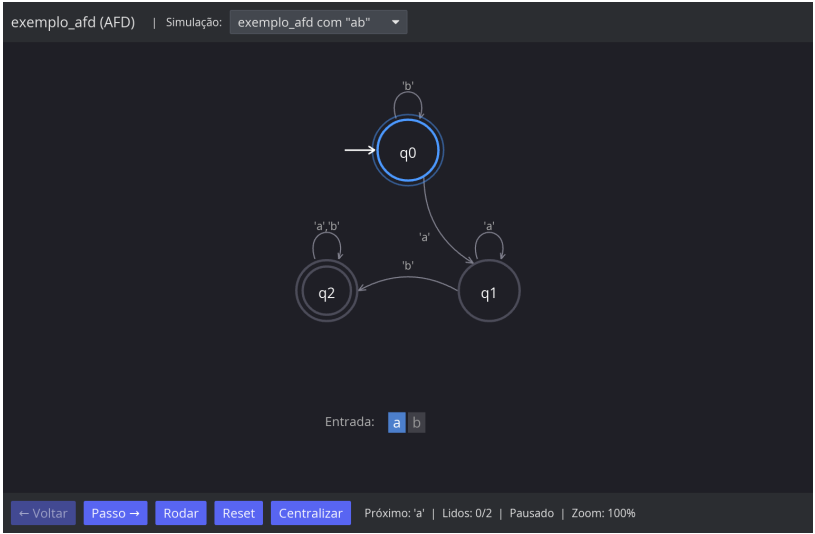
- Caractere `%` não reconhecido → *"token inválido"*.
- AFD com transição ε → *"AFD não pode ter epsilon"*.
- Falta a palavra `inicial` → *"esperava 'inicial' "*.

Substitui o "tentar e errar visual" por **retroalimentação imediata e precisa**.

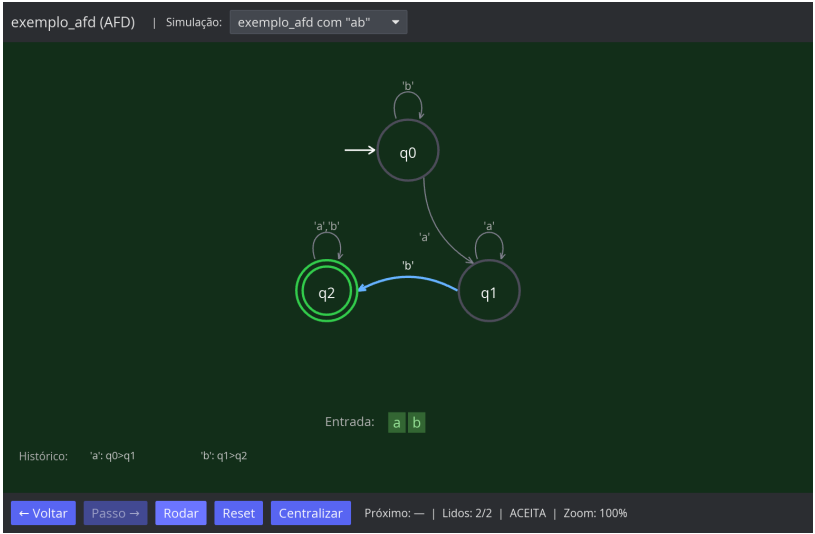
Roteiro

1. Introdução
2. Autômatos Finitos e Linguagens Formais
3. Compiladores e Linguagens de Domínio Específico
4. Rust como Linguagem de Implementação
5. Trabalhos Relacionados
6. Projeto da DSL
7. Implementação
8. Resultados
9. Conclusão

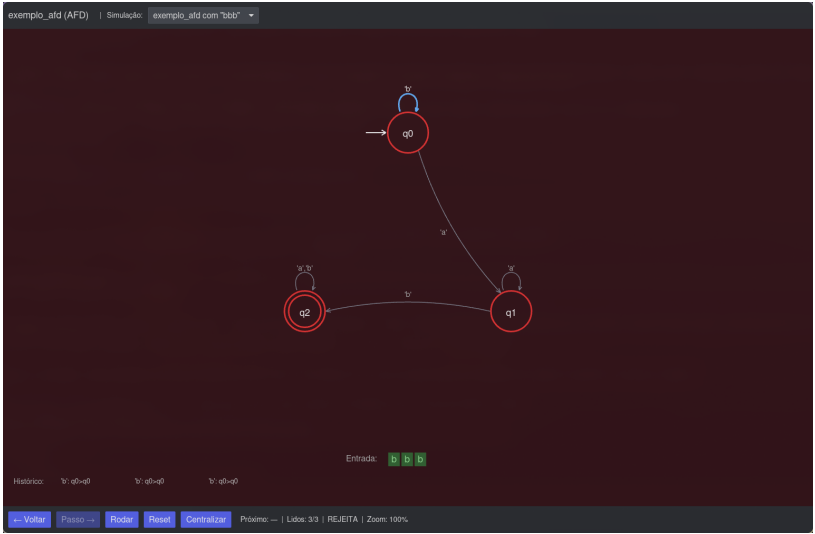
Interface — visão geral



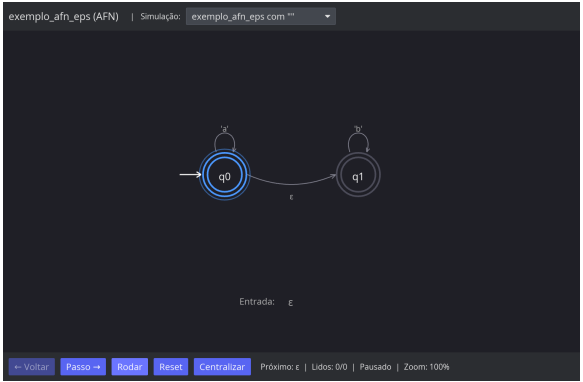
Simulação — veredito de aceitação



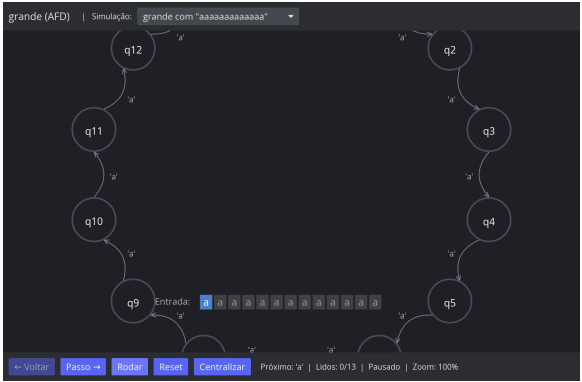
Simulação — veredito de rejeição



AFN-ε e navegação em diagramas grandes



AFN com transições ε



AFD de 14 estados — *pan* e *zoom*

Validação — testes automatizados

51 testes automatizados

Analisador léxico	12
Analisador sintático	15
Analisador semântico	15
Simulador	9
Total	**51**

- Cobrem reconhecimento de *tokens*, construção da AST e todas as verificações semânticas.
- **Todos os resultados** concordaram com o esperado.

Validação — simulações com autômatos clássicos

Autômatos de Hopcroft (2006) e Sipser (2012):

Autômato	**Cadeia**	**Esperado**	**Obtido**
AFD: contém 01	1101	Aceita	Aceita
AFD: contém 01	111	Rejeita	Rejeita
AFN: termina em 01	001	Aceita	Aceita
AFN: termina em 01	010	Rejeita	Rejeita
AFN-ε: a^*b^*	aabb	Aceita	Aceita
AFN-ε: a^*b^*	ba	Rejeita	Rejeita
AFN-ε: a^*b^*	ε	Aceita	Aceita

Todos os vereditos corretos $\Rightarrow$ **correção do simulador** (AFD e AFN-ε).

Validação — cenários de diagnóstico de erro

Arquivos com **erros propositais**, simulando o uso real por estudantes:

Arquivo	Tipo de erro	Diagnóstico
`sema_error.asl`	4 erros semânticos	4 mensagens localizadas
`sema_dfa_epsilon.asl`	AFD com ε	"AFD não pode ter epsilon"
`sema_nondeterministic.asl`	AFD não-determinístico	Transição conflitante
`parse_error.asl`	Falta `inicial`	"esperava 'inicial' "
`symbol_error.asl`	Caractere inválido	"token inválido"

Todas as mensagens com **linha, coluna, trecho realçado e descrição em português**.

Roteiro

1. Introdução
2. Autômatos Finitos e Linguagens Formais
3. Compiladores e Linguagens de Domínio Específico
4. Rust como Linguagem de Implementação
5. Trabalhos Relacionados
6. Projeto da DSL
7. Implementação
8. Resultados
9. Conclusão

Contribuições

A **Autosim** preenche a lacuna entre a descrição formal textual e a visualização gráfica:

- Uma **DSL** declarativa em português para AFDs e AFNs (com ε), formalizada em EBNF.
- Um **compilador pedagógico completo** (Logos + Chumsky + análise semântica) com **diagnósticos localizados e coloridos**.
- Um **simulador** com histórico e navegação bidirecional.
- Uma **interface gráfica** que renderiza e anima a simulação passo a passo.
- Validação com **51 testes** e simulações clássicas — **todos corretos**.

Limitações e trabalhos futuros

Limitações

- Alfabeto único por arquivo.
- Sem modo puramente textual (CLI).
- Um arquivo por execução (sem módulos).
- Sem editor embutido com realce.
- Sem conversão AFN→AFD / minimização.

Trabalhos futuros

- Editor com realce na própria interface.
- Alfabetos distintos por autômato.
- Conversão AFN→AFD, minimização e conversão para expressões regulares.
- Descrição e simulação de gramáticas formais.
- Modo CLI para automação.
- Autômatos com pilha e **máquinas de Turing** (níveis superiores da Hierarquia de Chomsky).
- **Estudos controlados com estudantes** (impacto pedagógico).

Obrigado!

Perguntas?

João Thomaz Vieira · Klaus Siegfried Beckmann

Orientador: Prof. Diógenes Cogo Furlan · UTP, 2026